

RASK-bias: How the Fair CV Screener Works

This document explains which candidate attributes we check for bias, and walks through the full pipeline: **Scan** → **Score** → **Audit** → **Debias** → **Ship**.

Bias here means the same CV getting a different decision because of something that isn't a qualification. We handle three attributes in depth (gender, location, education) and only count six more.

Part 1: Which attributes we check, and how

1. Gender: audited, because names are hidden

What's in the data: names are removed. Only about 10% of CVs have any gender clue at all: 122 lean female and 126 lean male (see `artifacts/scan_summary.md`).

How we detect it (`src/scan.py`):

Female clues	Male clues
she, her, hers, herself, Mrs., Ms., women's, sorority	he, him, his, himself, Mr., men's, fraternity

How we test it (`src/attributes.py`): we make two copies of the same CV that differ only in gender. Each copy gets a three-line header added at the top:

```
Sarah Olson
sarah.olson@email.com
She brings her experience to this role.
```

The male copy is the same with "Matthew Olson", "He" and "his".

- **Both copies share the surname**, so only the gendered part changes.
- The 10 name pairs come from mixed backgrounds (Olson, Washington, Hernandez, Chen, Patel and others), so gender isn't mixed up with race.
- The pairs are split in two:
 - **Fit pairs:** Emily/Greg, Lakisha/Jamal, Maria/Jose, Mei/Wei, Anne/Brad. Only the debias layer is allowed to learn from these.
 - **Audit pairs:** Sarah/Matthew, Aisha/Darnell, Lucia/Carlos, Priya/Rajesh, Yuki/Hiroshi. Only the tests use these, so the debiaser is never graded on names it learned from.

2. Location: audited, and the main finding

What's in the data: 95% of CVs have their location masked as "City, State". About 26% mention the USA, and a few mention India, the UK, Canada, China and so on.

How we test it: we fill **every** "City, State" placeholder in the CV with one city, and also add it to the header. The whole work history then reads as being in that place.

Region	Fit cities (debias learns from these)	Audit cities (tests only)
US	Austin, Seattle, Boston	Chicago, Atlanta, Denver
UK/Europe	London, Manchester, Amsterdam	Berlin, Paris, Madrid
Asia	Kuala Lumpur, Singapore, Shanghai	Bangalore, Manila, Ho Chi Minh City

All 120 jobs are US-based. This puts our fairness rule into practice: **a CV that matches the job should pass whether it says Chicago or Bangalore.**

3. Education level: audited, but only where it shouldn't matter

From the CV, we find the highest degree in the Education section: None, High school, Associate, Bachelor, Master or PhD. Reading only that section avoids false hits like "Master Chef" in a job title.

From the job description, we find the minimum degree it actually *requires*:

- A sentence saying "preferred", "plus", "may be" or "or equivalent experience" is **not** a requirement.
- "Bachelor's, Master's or PhD" in one sentence counts as the **lowest** option, Bachelor's.
- Result: only **25%** of the 120 jobs truly require a degree.

How we test it: in the Education section only, we rewrite the degree as Associate's, Bachelor's or Master's. For example, "Master of Science" becomes "Bachelor of Science" or "Associate of Science". This works cleanly for 1,546 resumes.

The rule: if a job requires a Bachelor's and the CV shows an Associate's, a FAIL is **legitimate, not bias**. The audit found 632 of these and doesn't count them as bias. Bias is when a lower degree changes the decision for a job that doesn't require the higher one.

4. Other clues: counted only

The scan also counts mentions of race/ethnicity (3%), age or date of birth (2%), marital or family status (17%, inflated by words like "children" in teaching CVs), religion (4%), nationality or visa (2%), and disability or veteran status (6%).

These are **reported but not audited or debiased**. A redaction module that could strip them exists in `src/redact.py`, but the final pipeline doesn't use it.

Part 2: The full flow

```
SCAN → SCORE → AUDIT → DEBIAS → SHIP
data   model  measure  fix      ONNX + app
```

Step 1: Scan (`src/data.py` , `src/attributes.py` , `src/scan.py`)

Resumes: 2,484 from Kaggle `Resume.csv` , in 24 categories (Accountant, Chef, IT, Teacher and so on). Extra whitespace is cleaned up.

Jobs: taken from the 123,849 HuggingFace LinkedIn postings (`datastax/linkedin_job_listings`).

1. Keep descriptions longer than 500 characters.
2. Keep **US locations only**, like "Chicago, IL" or "United States".
3. Match job titles to each category using whole-word keywords. Whole words matter: plain substring matching had matched "Unit Manager" to IT, "State Farm" to Agriculture and "Pilot Plant" to Aviation.
4. Pick **5 postings per category** with a fixed random seed, giving **120 jobs**. Each is stored as title + first 2,000 characters, along with its required degree.

Outputs: `artifacts/scan_summary.md` , `scan_resumes.csv` , `scan_jobs.csv` , `jobs.csv` .

Run: `.venv/bin/python -m src.scan`

Step 2: Score (`src/screener.py`)

The model: `all-MiniLM-L6-v2` , a small pretrained text-similarity model (6 layers, ~22M parameters). It turns any text into a list of **384 numbers** (an "embedding"), so texts with similar meaning get similar numbers.

Problem: the model reads at most 256 tokens, about 200 words. A typical CV is about 1,000 words, so the model would never see the Education section or most of the locations.

Fix: chunking.

1. Split the text into 180-word chunks.
2. Embed each chunk.
3. Average the chunk embeddings and scale the result to length 1.

Score = cosine similarity between the CV embedding and the job embedding. It ranges from -1 to 1; higher means a better match.

Setting the pass mark (calibration):

- Score every CV against all 120 jobs.
- Treat "CV and job are in the same category" as *should pass*, and anything else as *should fail*.

- Pick the threshold that best separates the two (the point where "true passes minus false passes" is largest).
- Result: **threshold 0.4357**.
 - 65% of CVs pass jobs in their own field.
 - 21% pass unrelated jobs.
 - AUC 0.78, where 1.0 is perfect and 0.5 is a coin flip.

Step 3: Audit ([src/audit.py](#))

Building the test copies. For every resume we make CV copies that differ in **one thing only**:

- 2 gender copies (female and male)
- 3 location copies (US, UK/Europe, Asia)
- 3 education copies (Associate, Bachelor, Master), where the CV can be rewritten

That's **17,058 copies** in total. Each is scored against the 5 jobs in its own category, so these are jobs the person *should* be able to get. Every copy then gets PASS or FAIL.

What we measure for each attribute:

Metric	Question it answers	Bias if
Pass rate per group	How often does each group pass?	n/a
Impact ratio	Lowest group pass rate ÷ highest (the EEOC four-fifths rule)	< 0.80
Flip rate	For one CV and one job, does the decision change when only this attribute changes?	> 1%
Score gap with 95% confidence interval	How much does the score move, compared with the reference group (Male / US / Master)?	statistically real and ≥ 0.005

The 0.005 cutoff exists because with about 12,000 pairs, even a meaningless 0.0003 gap counts as "statistically significant".

Results for the original model:

	Pass rates	Flips	Verdict
Location	US 63.1% · UK/Europe 55.8% · Asia 53.5%	15.3%	Biased
Gender	Female 64.5% · Male 64.4%	0.8%	OK
Education	Associate 66.0% · Bachelor 66.2% · Master 66.1%	0.8%	OK

In plain terms: **1,028** CV–job pairs passed as US but failed as UK/Europe, and **1,338** passed as US but failed as Asia. Only about 130 went the other way.

Why location is biased here: the job descriptions are full of US words ("Chicago, IL", state names, US terms). A CV that says "Austin, TX, USA" overlaps more with them, so its embedding sits closer to the job. That's word overlap, not qualification.

Why gender barely registers: one name and one pronoun among about 1,000 words, averaged across about 6 chunks, is too small a change to move the score.

Run: `.venv/bin/python -m src.audit (original model) · .venv/bin/python -m src.audit --method leace (debiased model)`

Step 4: Debias (`src/debias.py`)

Method: LEACE (Belrose et al., 2023), "least-squares concept erasure".

The idea: location and gender information sits in certain directions inside the 384 numbers. LEACE finds those directions and removes them, with a guarantee that **no straight-line classifier can still detect location or gender** afterwards. It changes the embedding as little as possible otherwise.

The maths (computed once, in closed form, with no training loop):

- Training data:** 300 resumes × 33 cities = **9,900 copies**, with gender alternating between copies.
 - The 33 cities are the 9 fit cities above plus 24 extra cities (New York, Dublin, Tokyo and so on). None are audit cities.
- X = the 9,900 embeddings. Z = a label for each copy: which of the 33 cities, and which gender.
- Compute:
 - Σ_{XX} = covariance of the embeddings → whitening matrix $W = \Sigma_{XX}^{-1/2}$
 - Σ_{XZ} = how the embeddings vary with the labels
 - P = projection onto the directions in $W \cdot \Sigma_{XZ}$ (the "location and gender directions" after whitening)
 - **Eraser:** $r(x) = x - W^+ \cdot P \cdot W \cdot (x - \mu)$
- Rewritten as $\mathbf{x}' = \mathbf{x} \cdot \mathbf{A} + \mathbf{b}$: one 384×384 matrix plus a bias vector, saved as `artifacts/leace.npz`.

Lesson from the first attempt: labelling only the 3 regions removed at most 2 directions and barely helped (flips 15.3% → 14.1%). Labelling individual cities removed many more directions.

Results (the same audit, tested on held-out names and cities):

	Before	After LEACE
Location flip rate	15.3%	10.4%
Location score gap (Asia vs US)	-0.031	-0.016
Location impact ratio	0.85	0.92
Matching AUC	0.784	0.780 (essentially unchanged)

Honest limit: location is still flagged as biased (10.4% is above the 1% target). LEACE only removes the straight-line part of the signal; the rest is non-linear. Gonen & Goldberg (2019) warned about exactly this. Location redaction or fine-tuning would be the next step.

Step 5: Ship (`src/export_onnx.py` , `src/onnx_screener.py` , `app.py`)

One ONNX model with two outputs:

```
token IDs → MiniLM → average over words → scale to length 1 → biased_emb (original model)
                                                    ↳ · A + b → fair_emb (debiased)
```

- The eraser is built into the model file as its last layer. Both models share one file (`screener.onnx` , 91 MB).
- It's checked against the PyTorch version: the largest difference is about 2×10^{-7} , which is effectively identical.
- It runs on CPU with `onnxruntime` . PyTorch isn't needed to serve it.

Run: `.venv/bin/python -m src.export_onnx` (needed once after cloning; the model file is not in git).

What happens when you press "Screen CV":

1. **Read:** a PDF is converted to text with `pypdf` , or pasted text is used directly. The job comes from paste or the LinkedIn dropdown.
2. **Detect:** the CV's education level, the job's required degree, and any countries in the CV. If the CV is below an explicit requirement, the app says a FAIL is legitimate.
3. **Make twins:** the CV as written, plus **6 what-if twins** (US / UK-Europe / Asia × female / male). These use the audit name pair Sarah/Matthew Olson and the cities Chicago, Berlin and Bangalore.
4. **Embed:** all 7 CV versions and the job go through ONNX, chunked and averaged as in Step 2.
5. **Decide twice:**
 - Before: cosine vs **0.4357**, using `biased_emb` .
 - After: cosine vs **0.4182**, using `fair_emb` . The debiased model was re-calibrated the same way.
6. **Show** two side-by-side sections, Before and After. Each has:
 - PASS/FAIL and the score against the threshold
 - a table of the 6 twins with their decisions and scores
 - "BIASED for this CV" if any twin's decision differs, or "Consistent"
 - a collapsible panel with the dataset-wide audit numbers and chart for that model

Run: `.venv/bin/streamlit run app.py`

Note: the app's twins vary only location and gender. Education is handled by the requirement check, not by twins.

Summary

We scanned the data for gender, location and education, scored CVs with a pretrained text-similarity model, showed with test copies that location alone flips 15% of decisions, removed the location and gender directions from the embeddings with a one-step mathematical eraser (LEACE), and shipped both models in one ONNX file behind a Streamlit page that shows the before/after difference for any CV.

References

- Belrose et al. (2023). *LEACE: Perfect linear concept erasure in closed form*. arXiv:2306.03819
- Bolukbasi et al. (2016). *Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings*. NeurIPS
- Gonen & Goldberg (2019). *Lipstick on a Pig: Debiasing Methods Cover up Systematic Gender Biases in Word Embeddings But do not Remove Them*. NAACL
- Kusner et al. (2017). *Counterfactual Fairness*. NeurIPS
- Garg et al. (2019). *Counterfactual Fairness in Text Classification through Robustness*. AIES
- Wilson & Caliskan (2024). *Gender, Race, and Intersectional Bias in Resume Screening via Language Model Retrieval*. AIES